DEVELOPER · PORTFOLIO 2026

# Kang-Hyun Kim

김 강 현

**제품을 끝까지 직접 만드는** 개발자입니다.

React · Next.js 프론트엔드부터

NestJS · AI **에이전트** 백엔드까지,

클린 아키텍처로 **실제 운영하는 서비스**를 만듭니다.

REACT / NEXT.JS | TYPESCRIPT | NODE.JS / NESTJS | AI / LLM AGENTS | CLEAN ARCHITECTURE | 8 PROJECTS

EMAIL dev.kanghyun@gmail.com

GITHUB github.com/developer-kanghyun

TECHNICAL STACK

# Skills

## FRONTEND

React · Next.js · TypeScript · TanStack Query · Zustand · React Konva · Tailwind CSS · Styled-Components · Vite

## BACKEND

NestJS · Node.js · Spring Boot 3 · Java 21 · WebFlux · Socket.IO · REST API

## DATABASE & INFRA

PostgreSQL · Prisma · Redis · Supabase · AWS S3 · Docker · Nginx · GCP

## DEVOPS & TESTING

GitHub Actions · Vercel · Docker Compose · CI/CD · Playwright · Jest · Web Push · PWA

## AI / LLM ENGINEERING

Claude API · Tool Calling · ReAct Agent · Function Calling · pgvector / RAG · LLM Fallback · Prompt Engineering · BullMQ · Whisper · Gemini

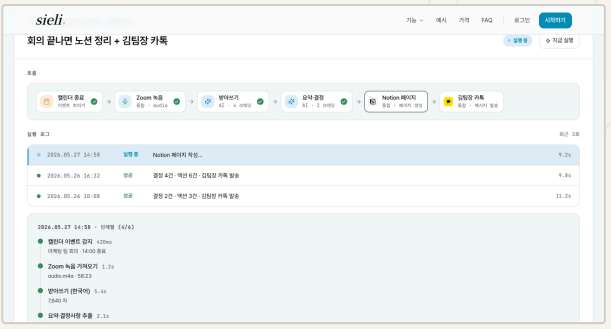
PROJECT 01 · SOLO · FULL-STACK + AI · 실서비스 운영

# Sieli

한국 직장인용 AI 업무 비서 — 자율 에이전트 · 자연어 자동화

LIVE DEMO




## OVERVIEW

기획·디자인·프론트·백엔드·인프라를 혼자 구축해 실제 운영 중인 (sieli.ai) AI 어시스턴트. 메일·회의·문서 등 매일 쓰는 업무를 처리하고, 자연어로 만든 자동화를 자율 에이전트가 알아서 실행합니다. Next.js 15 + NestJS 11(Clean Architecture + AI Layer) 모노레포.

## KEY ARCHITECTURE

### AUTONOMOUS AGENT RUNTIME

Claude tools API(function calling) + ReAct 루프 + pgvector 메모리 회상으로 목표 기반 자율 실행. 발송·결제 등 위험 작업은 멈추고 승인 요청(human-in-the-loop).

### DISTRIBUTED-SAFE EXECUTION

updateMany 조건부 원자 클레임으로 멀티 인스턴스에서 같은 작업 중복 실행 차단. 외부 분산락 인프라 없이 DB 한 줄로 race condition 해결.

## TECH STACK

Next.js 15 · NestJS 11 · TypeScript · Prisma · PostgreSQL · Redis · BullMQ · Claude API · Gemini · pgvector · Cloudflare R2 · Vercel · Fly.io

## KEY FEATURES

- ◇ 9가지 업무 카드 (회의 요약·계약서 검토·보고서·메일 교정)
- ◇ 자연어 → 자동화 DSL 변환 (멀티 LLM 폴백 + 방어 파싱)
- ◇ 이벤트 기반 이메일 트리아지 → 답장 초안 + 승인 발송
- ◇ 카카오 메시징 + 데스크톱(Tauri)·브라우저 확장 앱
- ◇ 크레딧 기반 과금 (PortOne v2) · 플랜별 사용량 게이팅

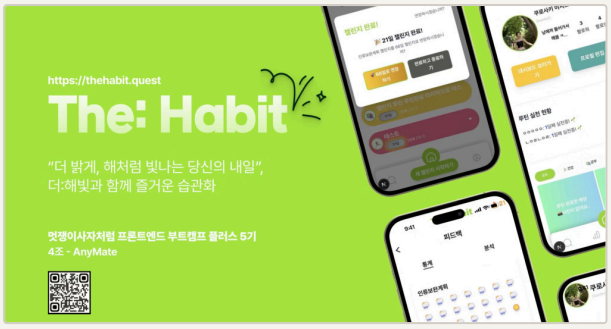
FULL-STACK · AI ENGINEERING · DEVOPS · SOLO DEV

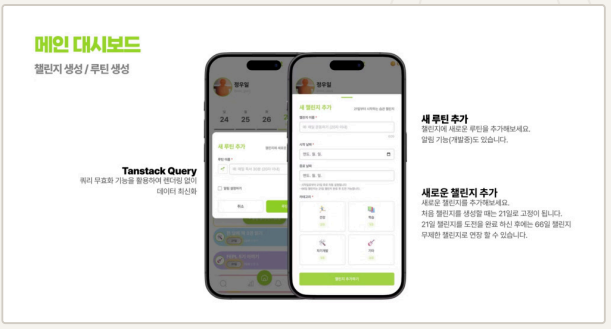
PROJECT 02 · TEAM 5인 · FULL-STACK + DEVOPS

# The Habit

PWA 기반 습관 트래커 서비스

GITHUB




## OVERVIEW

모바일 퍼스트 PWA로 제작한 습관 트래커 서비스. 오프라인에서도 동작하며 Web Push 알림으로 리마인드 기능을 제공합니다. Clean Architecture를 적용하여 비즈니스 로직과 UI를 완전히 분리했습니다.

## KEY FEATURES

- ◇ 습관(루틴) 관리 및 달성 기록 캘린더
- ◇ Web Push 기반 루틴 알림
- ◇ PWA 설치 지원 (모바일 홈 화면 추가)
- ◇ NextAuth.js 소셜 로그인 (Google, GitHub)
- ◇ Clean Architecture — Domain / Application / Infrastructure 분리

## MY CONTRIBUTIONS

- ◇ 루틴·루틴 완료 CRUD 로직 및 API (Clean Architecture 적용)
- ◇ Prisma ERD 설계 주도 (알림 테이블 직접 설계·추가)
- ◇ PWA 웹 푸시 알림 시스템 (알림 페이지, 루틴 시간 기반 발송)
- ◇ 루틴 완료 인증샷 AWS S3 업로드
- ◇ GitHub Actions CI/CD 및 서버 배포 파이프라인 구축
- ◇ 커스텀 도메인 구매 및 DNS 설정

## TECH STACK

Next.js · React · TypeScript · Prisma · PostgreSQL · TanStack Query · Zustand · NextAuth.js · AWS S3 · Web Push · GitHub Actions · PWA

## HIGHLIGHTS

**CLEAN ARCHITECTURE**

루틴·루틴 완료 도메인을 Domain / Application / Infrastructure로 나눠 구현. 의존성이 도메인 쪽으로만 향하도록 정리.

**WEB PUSH 알림**

Web Push API로 알림 구독·해제와 발송 구조 구현. 사용자가 설정한 루틴 알림 시간에 맞춰 푸시 발송.

**CI/CD & 도메인**

GitHub Actions에서 빌드 후 SSH로 서버에 접속해 PM2로 실행하는 배포 파이프라인 구축. 커스텀 도메인 구매·DNS 설정.

● ○ ○ ○ ○ 5인 팀 프로젝트

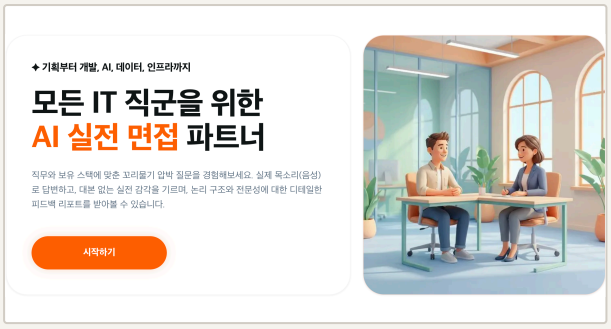
FULL-STACK · DB DESIGN · CI/CD

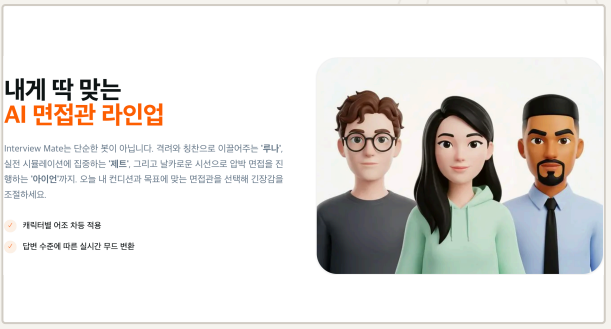
PROJECT 03 · SOLO · FULL-STACK · 샤이닝라이언 인턴

# Interview Mate

AI 면접 연습 플랫폼

LIVE DEMO | GITHUB




## OVERVIEW

AI 면접관과 실전처럼 면접 연습을 할 수 있는 풀스택 플랫폼. Next.js 14 + Spring Boot 3.2 구성. FSD + Clean Architecture에 AI 레이어를 결합한 독자적 구조로 설계했습니다. 전체 개발을 TDD로 진행했습니다.

## ARCHITECTURE

- ◇ FSD + Clean Architecture + AI Layer 3계층 결합 설계
- ◇ AI 레이어 — 프롬프트 관리·모델 추상화·응답 스트리밍 분리
- ◇ TDD — Red-Green-Refactor 사이클로 전체 개발 진행
- ◇ Testcontainers로 실제 PostgreSQL 환경 통합 테스트
- ◇ Redis (Caffeine + Redis) 2-tier 캐싱

## TECH STACK

Next.js 14 | Spring Boot 3.2 | Java 21 | PostgreSQL | Redis | WebFlux | Playwright | Testcontainers

## HIGHLIGHTS

### FSD + CLEAN + AI LAYER

Feature-Sliced Design에 Clean Architecture와 AI 레이어를 결합. 높은 응집도·낮은 결합도.

### TDD DEVELOPMENT

전체 개발을 TDD로 진행. Red-Green-Refactor 사이클로 안정적인 코드 품질 확보.

### 2-TIER CACHING

로컬 Caffeine + 분산 Redis 2단계 캐싱으로 반복 요청 최적화.

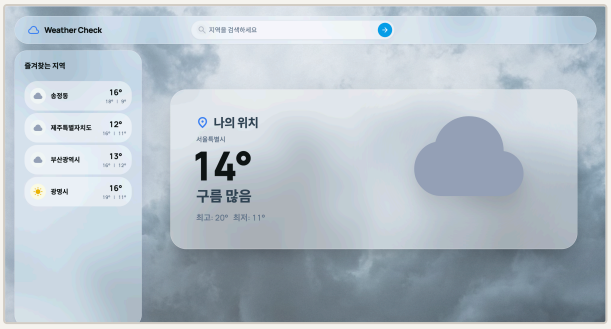
FULL-STACK | SOLO DEV

PROJECT 04 · SOLO · FULL-STACK

# Weather Check

한국 지명 기반 날씨 조회 서비스

LIVE DEMO GITHUB







## OVERVIEW

브라우저 Geolocation API로 좌표를 수집하고, 역지오코딩으로 한국 행정 지명을 추출한 뒤 날씨 API에 전달하는 3단계 파이프라인을 직접 설계. 외국 API의 한국 지명 인식 한계를 Fallback 전략으로 해결하고, Context API로 Props Drilling 문제를 해소했습니다.

## PROBLEM SOLVED

### 3-TIER KOREAN SEARCH FALLBACK

OpenWeatherMap의 한국어 지명 미인식 문제를 "강남구" → "서울 강남" → "Gangnam Seoul" 단계별 재시도 전략으로 해결. API 인식 패턴을 분석해 쿼리 포맷을 직접 설계.

### CONTEXT API — PROPS DRILLING 해소

날씨·즐겨찾기·검색 상태가 3~4단계 drilling되는 문제를 외부 라이브러리 없이 Context API로 해결. 서버 상태(TanStack Query) / 클라이언트 상태(Context) 역할을 명확히 분리.

## TECH STACK

React 18 · TypeScript · TanStack Query v5 · Tailwind CSS · Context API · React Router v6 · Vite

## KEY FEATURES

- ◇ Geolocation → 역지오코딩 → 날씨 API 3단계 파이프라인 설계
- ◇ 한국 지명 자연어 검색 + 3단계 Fallback 인식 전략
- ◇ Context API로 Props Drilling 해소 (외부 라이브러리 없이)
- ◇ Optimistic UI — 즐겨찾기 즉시 반영 (API 응답 전 UI 업데이트)
- ◇ TanStack Query v5 stale-while-revalidate 캐싱

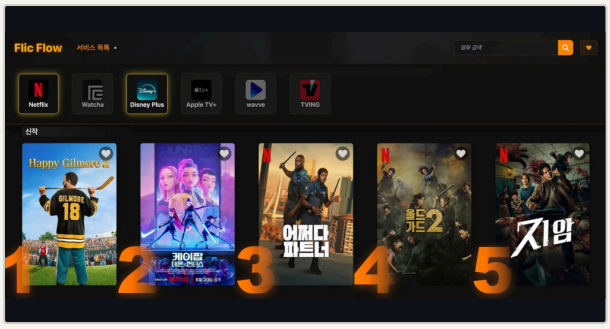
FULL-STACK SOLO DEV

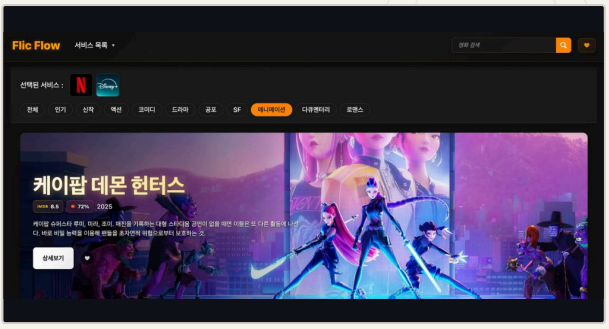
PROJECT 05 · SOLO · FRONTEND + DEVOPS

# Flic Flow

OTT 통합 검색 최적화 서비스

LIVE DEMO | GITHUB




## OVERVIEW

넷플릭스·왓챠 등 여러 OTT 플랫폼의 콘텐츠를 통합 검색하는 서비스.
클라이언트 사이드 캐싱과 API 최적화로 성능을 크게 향상시켰습니다.

## PERFORMANCE RESULTS

| 44% | 90% | 1.8s |
|---|---|---|
| 성능 향상 | API 호출 감소 | FCP (3.2s→1.8s) |

## KEY FEATURES

- ◇ OTT 통합 검색 및 필터링 (장르·플랫폼·평점)
- ◇ React Query 캐싱으로 중복 API 호출 90% 제거
- ◇ Intersection Observer 기반 무한 스크롤
- ◇ Docker + Nginx 컨테이너 배포

## TECH STACK

React | TypeScript | React Query | Vite | Docker | Nginx

## HIGHLIGHTS

**CACHING STRATEGY**

stale-while-revalidate로 네트워크 요청 최소화.
API 호출 90% 감소.

**BUNDLE OPTIMIZATION**

Route-level code splitting + dynamic import로
FCP 3.2s → 1.8s.

FRONTEND | DEVOPS | SOLO DEV

PROJECT 06 · SOLO · FRONTEND + DESIGN

# Portfolio

개인 포트폴리오 웹사이트

LIVE DEMO GITHUB

## OVERVIEW

React와 TypeScript로 제작한 개인 포트폴리오 웹사이트. 스크롤 기반 애니메이션과 인터랙티브한 UI를 통해 프로젝트와 기술 스택을 효과적으로 소개합니다.

## KEY FEATURES

- ◇ Intersection Observer로 섹션 진입 시 reveal 애니메이션 트리거
- ◇ CSS Custom Properties 기반 디자인 토큰 시스템 직접 설계
- ◇ --scroll-progress 변수 실시간 갱신으로 배경 기하 도형 CSS 애니메이션 구동
- ◇ useModal Generic 커스텀 훅으로 재사용 가능한 모달 시스템 설계
- ◇ MutationObserver로 동적 생성 요소의 hover 감지 (커스텀 커서)

## TECH STACK

React | TypeScript | Styled-Components | Vite | Intersection Observer | CSS Animation

## HIGHLIGHTS

### CSS DESIGN TOKEN SYSTEM

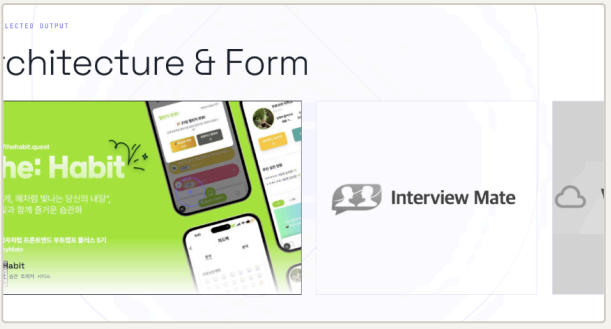
외부 UI 라이브러리 없이 CSS Custom Properties로 색상·폰트·간격 토큰을 직접 설계. scroll-progress 변수로 배경 애니메이션 실시간 제어.

### USEMODAL GENERIC HOOK

모달 상태와 데이터를 Generic 타입 훅으로 추상화. 어떤 데이터 타입에도 재사용 가능한 모달 인터페이스 설계.

FRONTEND DESIGN SOLO DEV



PROJECT 07 · SOLO · FULL-STACK + DEVOPS

# High-Yield Trading

실시간 주식 자동매매 풀스택 플랫폼



| 아이디 | 순위 | |
|---|---|---|
| 7kk***** | 12위 | ▲ 6 |

| 총손익 | 총평가금액 | 약정금액 |
|---|---|---|
| 35,373,193 | 404,467,193 | 131,619,389 |

## OVERVIEW

KIS(한국투자증권) API와 연동한 실시간 주식 자동매매 시스템. 멀티 브로커 어댑터 패턴으로 브로커 교체가 유연하며, Socket.IO 스트리밍으로 주가 변동을 즉시 반영합니다.

## KEY ARCHITECTURE

### MULTI-BROKER ADAPTER PATTERN

IBrokerAdapter 인터페이스 추상화 → KIS API 교체 시 비즈니스 로직 변경 없음

### REAL-TIME WEBSOCKET STREAMING

Socket.IO + Bull Queue로 주가 데이터 실시간 처리 및 자동매매 신호 발생

## KEY FEATURES

- ◇ 자동매매 전략 설정 및 실시간 포지션 모니터링
- ◇ KIS API 연동 — 실시간 체결가 스트리밍
- ◇ Docker Compose 멀티 서비스 오케스트레이션
- ◇ Prometheus + Grafana 모니터링 대시보드

## TECH STACK

Next.js 16 · NestJS 10 · TypeScript · PostgreSQL 16 · Prisma · Redis 7 · Bull Queue · Socket.IO · Docker Compose · Prometheus · Grafana · KIS API · GCP VM

## HIGHLIGHTS

### SYSTEM DESIGN

어댑터 패턴으로 브로커 독립성 보장. Bull Queue로 주문 처리 안정성 확보.

### MONITORING

Prometheus 메트릭 수집 + Grafana 시각화. GCP VM + Docker 프로덕션 배포.

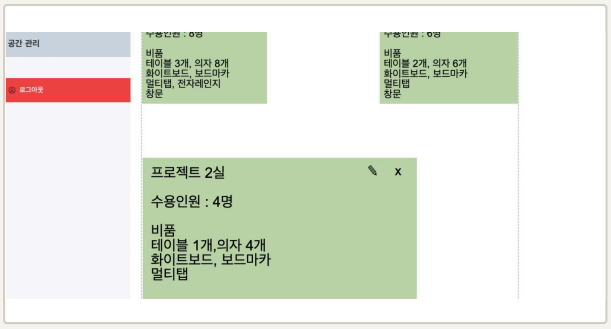
FULL-STACK · DEVOPS · SOLO DEV

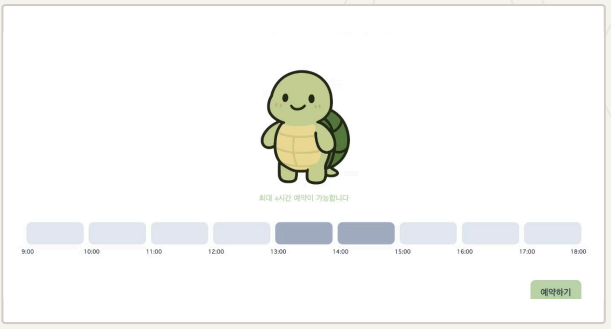
PROJECT 08 · TEAM 4인 · FULL-STACK

# TtaBook

Canvas 기반 비주얼 공간 예약 시스템

GITHUB




## OVERVIEW

React Konva(Canvas API)를 활용한 시각적 공간 예약 플랫폼. 사무실·스터디룸 등의 공간을 드래그&드롭으로 배치하고 실시간으로 예약 상태를 확인합니다. 4인 팀 프로젝트.

## KEY FEATURES

- ◇ React Konva Canvas 공간 배치 에디터
- ◇ 드래그&드롭으로 좌석·룸 시각적 배치
- ◇ Supabase Realtime 예약 상태 동기화
- ◇ Clean Architecture 레이어 분리

## MY CONTRIBUTIONS

- ◇ 관리자 공간 편집 캔버스 구현 (React Konva, 드래그&드롭 배치)
- ◇ 방 추가·삭제·위치 저장 기능 (UseCase · Supabase Repository)
- ◇ 공간 조회 · 방별 예약 조회 API 구현
- ◇ 팀 Clean Architecture 구조에 맞춘 공간·방 도메인 구현

## TECH STACK

Next.js 15 | React Konva | TypeScript | TanStack Query | Zustand | Supabase | CSS Modules

## HIGHLIGHTS

**CANVAS EDITOR**

관리자가 방을 드래그&드롭으로 배치·편집하는 캔버스 에디터를 React Konva로 구현.

**화면부터 API까지**

편집한 방 위치·구성을 저장하는 UseCase와 Supabase Repository를 직접 구현해 관리자 화면과 서버를 연결.

● ○ ○ ○ 4인 팀 프로젝트

FULL-STACK | CANVAS